CS 498 · Machine Learning Systems

PagedAttention

How can an LLM serve more requests with the same GPU memory?

KV caching · Memory fragmentation · Block-based serving

Fan Lai

University of Illinois Urbana-Champaign | Fall 2026

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy, and colleagues

Course reminders

Mid-semester project report

Due Wed, Nov 4

Four pages: problem, quantitative motivation, related work, approach, and evaluation.

Show measurable progress, not just a revised proposal.

Assignment 2

Due Mon, Nov 9

Attention, LoRA, PagedAttention, and MoE exercises

Dates from the accompanying Fall 2026 L15 slides; check Canvas for updates.

Today's lecture

From batching more requests to managing their KV state

01	**Recall**	Continuous batching and chunked prefill
02	**KV caching**	Save computation, but consume GPU memory
03	**PagedAttention**	Logical blocks, physical blocks, and block tables
04	**Evaluate**	Memory utilization, throughput, and open questions

Panel Discussion: DistServe

Learning objectives

By the end of class, you should be able to…

Explain	how KV caching reduces decoding compute while increasing memory demand.
Apply	a logical-to-physical block table across multiple decode steps.
Evaluate	fragmentation, sharing, overhead, and serving throughput tradeoffs.

Recall: continuous batching

Different requests enter and leave the decode batch over time.



Takeaways

Iteration-level batching reuses available slots as requests finish.

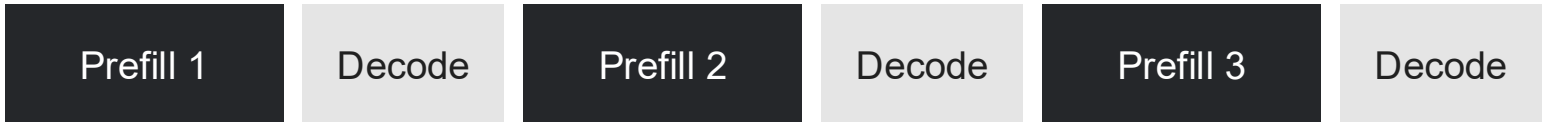
Recall: chunked prefill

Break a long prefill into smaller chunks and interleave decode.

Without chunking



With chunking



More concurrency is useful only if memory can hold the active states.

Recall: causal self-attention

Each query attends to earlier keys and mixes the corresponding values.

Q Current token queries

K Keys of past tokens

V Values of past tokens

Attention(Q, K, V)

$= \text{softmax}(QK^T / \sqrt{d})\, V$

Causality means a new token never changes the old K/V projections.

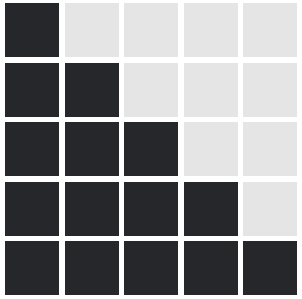
Prefill and decode have different shapes

Prefill handles a sequence; each decode step handles one new query.

PREFILL

Q: S tokens

K / V: S tokens



S × S causal attention

DECODE

Q: 1 token

K / V: all retained tokens



1 × S attention row

Without KV caching: repeated computation

If we recompute the whole prefix, every new output repeats old work.

Step 1 

Recompute K/V for 4 tokens

Step 2 

Recompute K/V for 5 tokens

Step 3 

Recompute K/V for 6 tokens

The cost of earlier tokens is paid again and again.

KV caching changes decode complexity

Compare attention work at position t (ignoring projection overhead).

Recompute full prefix	$O(t^2 \cdot H)$
Processes all earlier queries again	
Incremental KV decode	$O(t \cdot H)$
One new query attends to cached K/V	

KV caching exchanges repeated computation for persistent GPU memory.

GPU memory limits concurrent requests

Model weights, KV cache, and runtime buffers compete for the same space.



Illustration: FP16 13B parameters ≈ 26 GB before caches and buffers.

The practical KV budget is what remains after weight and runtime allocations.

Less memory waste → more live sequences → larger effective batch.

Naïve KV allocation: reserve the maximum

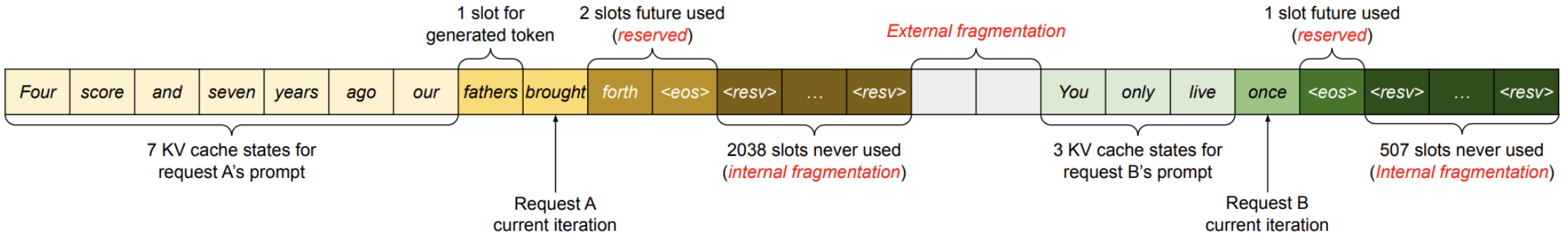
A request may be assigned one contiguous allocation for its longest possible output.



Dark: token KV allocated and used Gray: memory reserved but not yet needed

Output lengths are uncertain yet pre-reserving the maximum wastes capacity.

Three sources of KV-memory waste



01 **Reservation**

Memory reserved for tokens the request may never generate.

02 **Internal fragmentation**

Unused space within the last allocated storage unit.

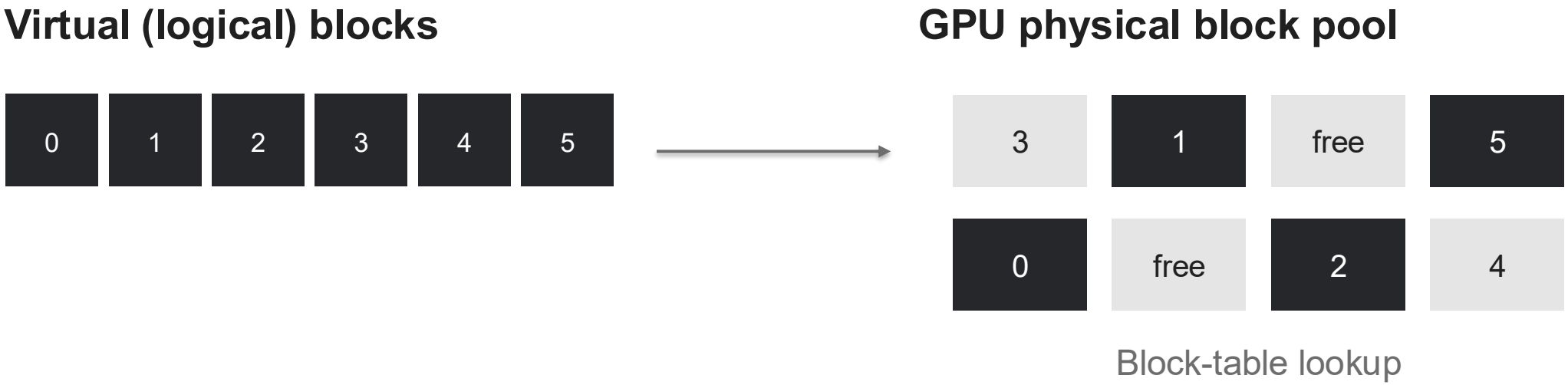
03 **External fragmentation**

Free space spread across gaps, blocking a large contiguous request.

Orca allocators use only about 20–40% for actual token states in these cases.

PagedAttention: The idea comes from OS memory paging

A logical sequence does not need a physically contiguous allocation.



Indirection lets requests grow without relocating their whole KV history.

PagedAttention: block-based KV storage

Split the KV cache into fixed-size blocks and allocate blocks on demand.

Logical blocks	Ordered by position within a request
Physical blocks	Placed anywhere in a GPU KV block pool
Block table	Maps logical block IDs to physical block IDs

The attention kernel must follow the block table when reading K/V.

Logical-to-physical block mapping

The same sequence order can be represented by scattered physical blocks.

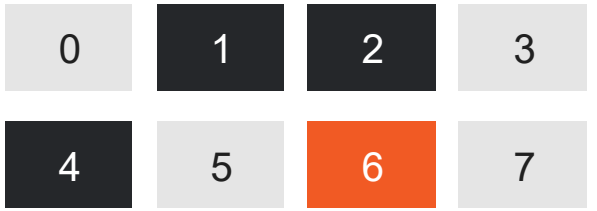
Logical sequence



Block table

0	4
1	1
2	6
3	2

Physical GPU pool

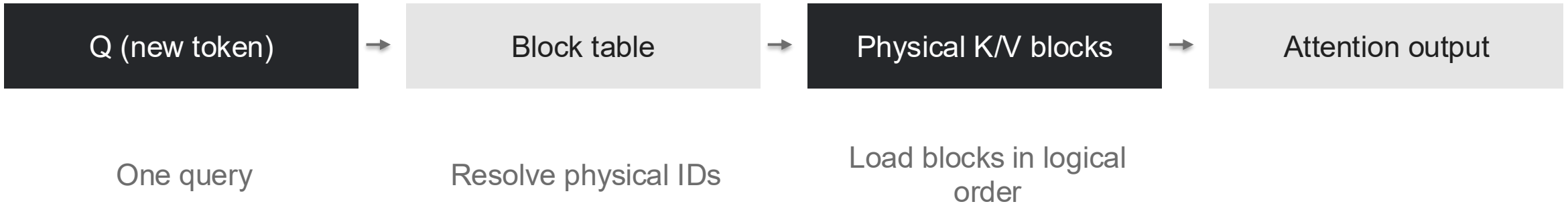


Mappings (0→4, 1→1, 2→6, 3→2) preserve logical token order.

Non-contiguous physical placement is invisible to the model.

PagedAttention still computes exact attention

Block indirection changes the read pattern, not the attention equation.



What changes? KV-address translation and block-aware memory access.

Paging is a memory-management mechanism, not token eviction.

Block table walkthrough: prompt allocation

Step 1

Prefill seven tokens; allocate logical blocks 0 and 1.

Logical tokens

A	B	C	D	E	F	G

Block size: 4 tokens

Block table

0	3
1	6

Physical KV blocks

0	1	2	3
4	5	6	7

Only the final block may be partially occupied.

Illustrative block size = 4 tokens; block IDs/placements are schematic.

Block table walkthrough: append in-place

Step 2

Decode token H into available space in logical block 1.

Logical tokens



Block size: 4 tokens

Block table

0	3
1	6

Physical KV blocks



Token H fills an already allocated physical block; no new block needed.

Illustrative block size = 4 tokens; block IDs/placements are schematic.

Block table walkthrough: cross a boundary

Step 3

Token I starts logical block 2; allocate one free physical block.

Logical tokens

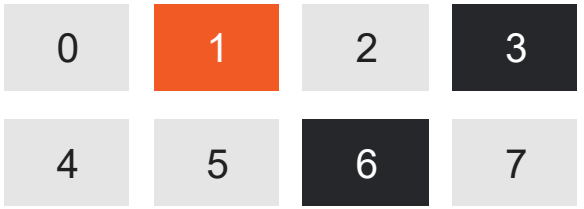


Block size: 4 tokens

Block table

0	3
1	6
2	1

Physical KV blocks



A new block is allocated only when the active logical block becomes full.

Illustrative block size = 4 tokens; block IDs/placements are schematic.

Block table walkthrough: next decode step

Step 4

Token J is written into the new block; existing blocks do not move.

Logical tokens

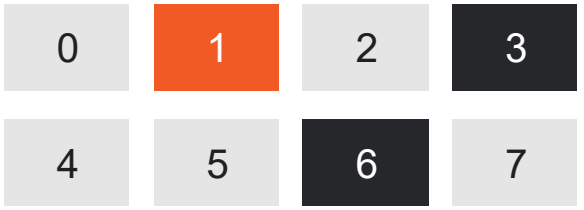


Block size: 4 tokens

Block table

0	3
1	6
2	1

Physical KV blocks



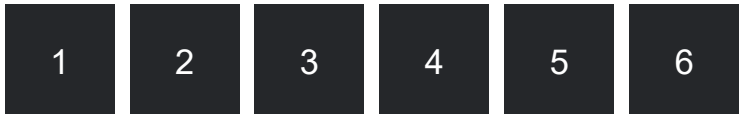
KV grows in small increments; previous blocks stay in place.

Illustrative block size = 4 tokens; block IDs/placements are schematic.

Multiple requests can share one block pool

Each request has its own logical view of the same GPU allocator.

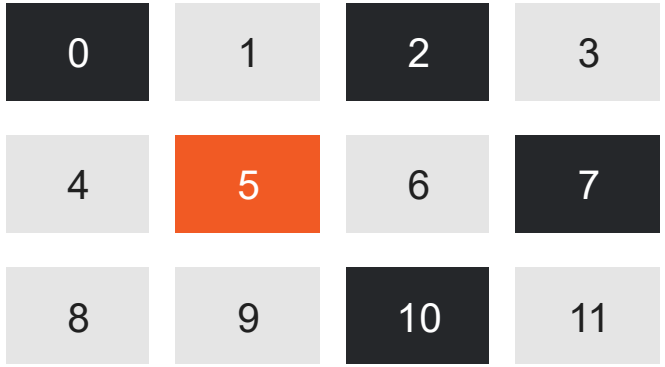
Request A



Request B



Shared physical block pool

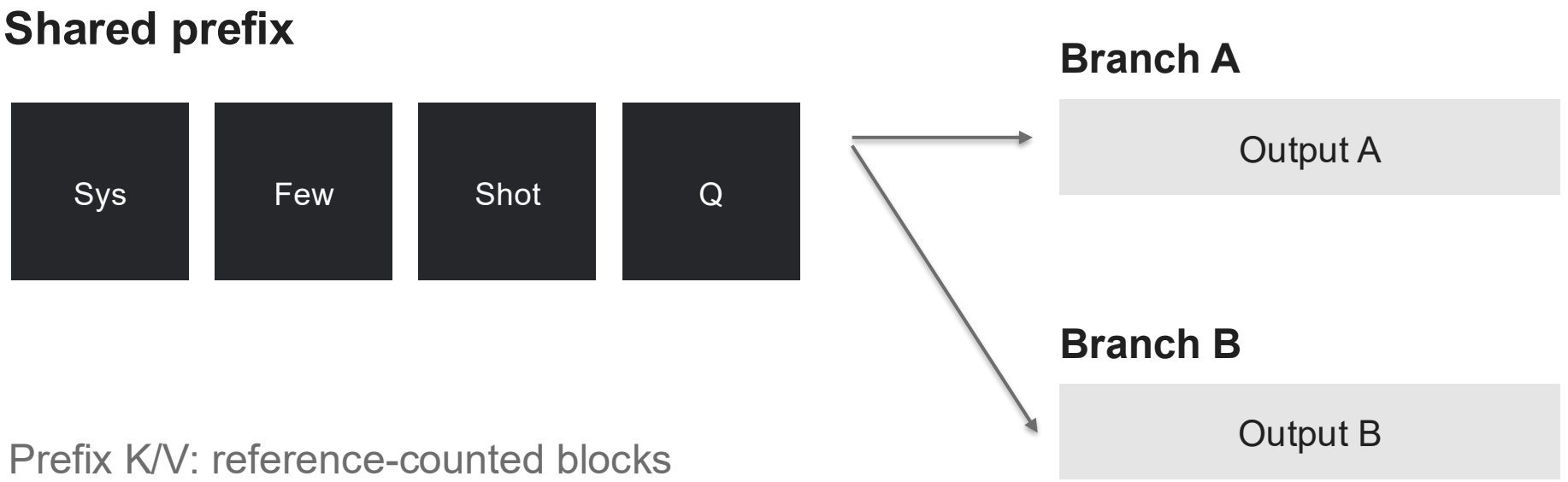


A: blocks 0, 7 | B: blocks 2, 5 | free: remaining blocks

A block returned by one request can be reused by another.

Paged blocks also enable copy-on-write sharing

Parallel outputs can share identical prefix KV until their histories diverge.



Prefix K/V: reference-counted blocks

Shared blocks save memory. Writes to a diverging block require isolation.

Paging bounds internal fragmentation

With B-token blocks, each sequence wastes fewer than B token slots.

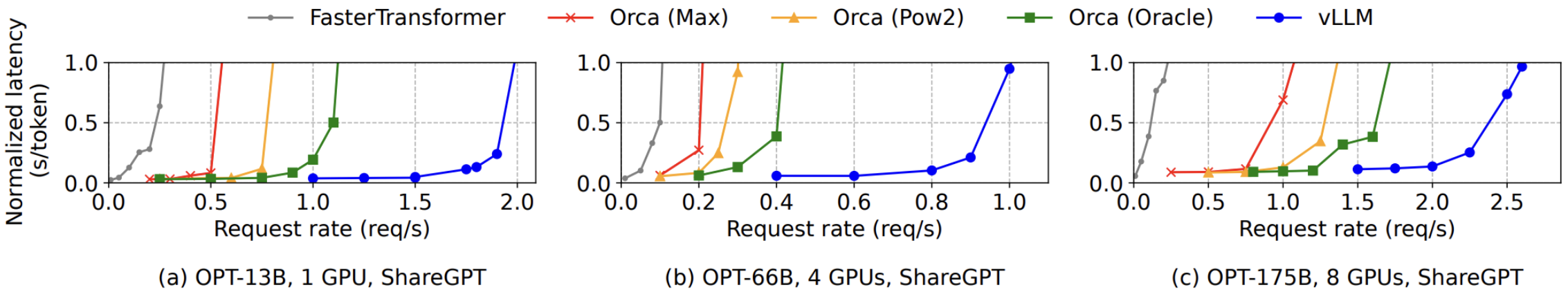


Example: B = 4 tokens per block, so waste per request ≤ 3 slots.

Unlike maximal contiguous reservation, unused capacity stays small.

vLLM: higher supported request rates

At comparable normalized latency, vLLM supports higher arrival rates.



(a) OPT-13B, 1 GPU, ShareGPT

(b) OPT-66B, 4 GPUs, ShareGPT

(c) OPT-175B, 8 GPUs, ShareGPT

The improvement comes from more usable KV memory and effective batching.

Result figures from the supplied L14 slides; Kwon et al., PagedAttention / vLLM, SOSP 2023.

What does paging cost?

01	**Block-table metadata**	Memory indirection and per-request block tracking.
02	**Kernel complexity**	Efficient attention across non-contiguous physical blocks.
03	**Allocator overhead**	Block allocation, release, reference counting, and copy-on-write.

Measure net serving throughput—not KV utilization alone.

Takeaways

KV caching saves decode computation.

But all retained tokens require persistent K/V storage.

PagedAttention eliminates contiguous sequence allocation.

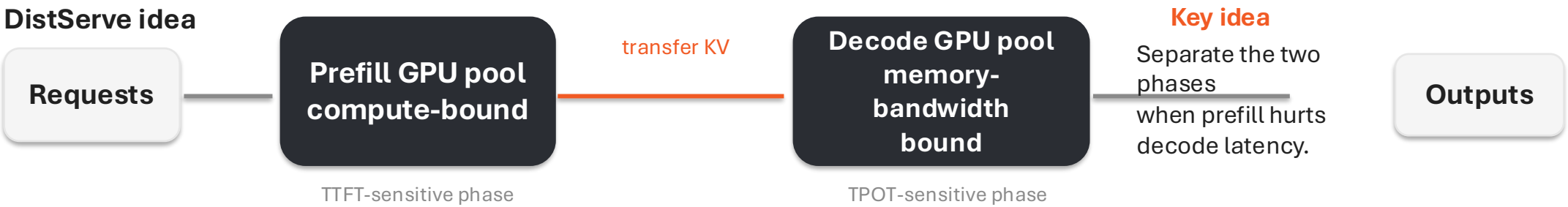
Logical blocks map to a shared pool of physical blocks.

Memory efficiency enables higher concurrency.

Savings must exceed bookkeeping and kernel overhead.

Systems connection: allocation efficiency and runtime scheduling are tightly coupled.

Panel preview: DistServe



Discuss: Should we optimize TTFT, TPOT, or service goodput? (Next lecture will answer it! ☺)

Connection: PagedAttention asks where KV is stored efficiently; DistServe asks where prefill and decode should run.

Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024.

References and acknowledgments

vLLM / PagedAttention	Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.
Orca / continuous batching	Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.
SGLang (panel)	Zheng et al. Efficient Execution of Structured Language Model Programs. NeurIPS 2024.
Next lecture	H2O (NeurIPS 2023) and StreamingLLM (ICLR 2024).

Thanks to Minjia Zhang, Arvind Krishnamurthy, and colleagues.